



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №5
на тему «ШАБЛони «ADAPTER», «BUILDER», «COMMAND»,
«CHAIN OF RESPONSIBILITY», «PROTOTYPE»»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мякий Михайло Юрійович

Київ 2024

Зміст

Тема	3
Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми	9
Реалізація шаблону за обраною темою	10
1. Command.....	10
2. CommandInvoker	11
3. InsertTextCommand.....	12
4. SaveCommand.....	13
5. FileController	14
6. EditService.....	15
7. FileService.....	16
Посилання на код на GitHub	17
Діаграма класів для патерну «Command»	18
Переваги застосування патерну «Command».....	18

ШАБЛони «ADAPTER», «BUILDER», «COMMAND», «CHAIN OF RESPONSIBILITY», «PROTOTYPE»

Мета: ознайомитися з шаблонами проектування «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та набути практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Хід роботи:

Короткі теоретичні відомості:

Анти-патерни (anti-patterns) — це погані рішення проблем, що часто використовуються в проектуванні. Вони є протилежністю «шаблонам проектування», які описують хороші практики. Анти-патерни в управлінні та розробці ПЗ включають:

Управління розробкою ПЗ:

- Дим і дзеркала: Демонстрація неповних функцій.
- Роздування ПЗ: Збільшення вимог до ресурсів у наступних версіях.
- Функції для галочки: Наявність непов'язаних функцій для реклами.

Розробка ПЗ:

- Інверсія абстракції: Приховування частини функціоналу.
- Невизначена точка зору: Моделі без чіткої специфікації.
- Великий клубок бруду: Система без чіткої структури.
- Бензинова фабрика: Необов'язкова складність дизайну.
- Затичка на введення даних: Недостатня обробка неправильного введення.

ООП:

- Базовий клас-утиліта: Спадкування замість делегування.
- Божественний об'єкт: Надмірна концентрація функцій в одному класі.
- Самотність: Надмірне використання патерну Singleton.

Анти-патерни в програмуванні:

- Непотрібна складність: Ускладнення рішень без необхідності.
- Дія на відстані: Несподівані взаємодії між різними частинами системи.
- Накопичити і запустити: Використання глобальних змінних для параметрів.
- Слепа віра: Недостатня перевірка результатів або виправлень.
- Активне очікування: Використання ресурсів під час очікування події замість асинхронного підходу.
- Кешування помилки: Забуте скидання прапора після обробки помилки.
- Перевірка типу замість інтерфейсу: Перевірка типу замість використання інтерфейсу.
- Кодування шляхом виключення: Обробка випадків через винятки.
- Потік лави: Залишення поганого коду через високу вартість його видалення.
- Спагеті-код: Заплутаний код з некерованим виконанням.

Методологічні анти-патерни:

- Програмування методом копіювання-вставки: Копіювання коду замість створення спільних рішень.
- Дефакторінг: Заміна функціональності документацією.
- Золотий молоток: Впевненість у застосуванні одного рішення для всіх проблем.
- Передчасна оптимізація: Оптимізація без достатньої інформації.
- Винахід колеса: Створення нових рішень замість використання існуючих.
- Винахід квадратного колеса: Погане рішення, коли є хороше.
- Самознищення: Помилка, що призводить до фатальної поведінки програми.

Анти-патерни управління конфігурацією:

- DLL-пекло: Проблеми з версіями і доступністю DLL.
- Пекло залежностей: Проблеми з версіями залежних продуктів.

Організаційні анти-патерни:

- Аналітичний параліч: Надмірний аналіз без реалізації.
- Дійна корова: Неінвестування в розвиток продукту.
- Тривале старіння: Перенесення зусиль на портинг замість нових рішень.
- Скидування витрат: Перенесення витрат на інші відділи.
- Повзуче покращення: Додавання змін, що знижують загальну якість.
- Розробка комітетом: Розробка без централізованого керівництва.
- Ескалація зобов'язань: Продовження хибного рішення.
- Розповзання рамок: Втрата контролю над проектом.
- Замикання на продавці: Залежність від одного постачальника.

Шаблон "Adapter" (Адаптер)

Призначення:

Адаптер використовується для приведення інтерфейсу одного об'єкта до іншого, забезпечуючи сумісність між різними системами.

Проблема:

Існує потреба адаптувати різні формати даних (наприклад, XML до JSON) без зміни вихідного коду.

Рішення:

Створення адаптера, який «перекладає» інтерфейси одного об'єкта для іншого (наприклад, XML_To_JSON_Adapter).

Приклад:

Адаптер для зарядки ноутбука в різних країнах, що дозволяє підключити пристрій до різних типів розеток.

Переваги:

Приховує складність взаємодії різних інтерфейсів від користувача.

Недоліки:

Ускладнює код через додаткові класи.

Шаблон "Builder" (Будівельник)

Призначення:

Шаблон відділяє процес створення об'єкта від його представлення, що зручно для складних або багатоформатних об'єктів.

Проблема:

Ускладнене створення об'єкта, наприклад, формування відповіді web-сервера з кількох частин (заголовки, статуси, вміст).

Рішення:

Кожен етап створення об'єкта абстрагується в окремий метод будівельника, що дозволяє контролювати процес побудови.

Приклад:

Будівництво будинку за етапами, де кожен етап виконується окремою командою будівельників.

Переваги:

Забезпечує гнучкість та незалежність від внутрішніх змін у процесі створення.

Недоліки:

Клієнт залежить від конкретних класів будівельників, що може обмежити можливості.

Шаблон "Command" (Команда)

Призначення:

Перетворює виклик методу в об'єкт-команду, що дозволяє гнучко керувати діями (додавати, скасовувати, комбінувати команди).

Проблема:

Як організувати обробку кліків у текстовому редакторі без дублювання коду для різних кнопок.

Рішення:

Створити окремий клас-команду для кожної дії, який буде викликати методи бізнес-логіки через об'єкт інтерфейсу.

Приклад:

Офіціант у ресторані приймає замовлення (команда) і передає кухарю для виконання (отримувач), без прямого контакту між клієнтом і кухарем.

Переваги:

- Знімає залежність між об'єктами, що викликають і виконують операції
- Дозволяє скасовувати, відкладати та комбінувати команди

- Підтримує принцип відкритості/закритості

Недоліки:

- Ускладнює код додатковими класами

Шаблон "Chain of Responsibility"

Призначення:

Ланцюг відповідальності дозволяє організувати обробку запиту, передаючи його послідовно через ланцюг об'єктів, поки не буде знайдено обробник, здатний виконати запит. Це зручно для побудови гнучкої системи обробників, коли важливо зменшити залежність клієнта від обробників і структурувати обробку.

Проблема:

Припустимо, потрібно реалізувати систему обробки онлайн-замовлень, де доступ мають тільки авторизовані користувачі, а адміністратори мають додаткові права. Ці перевірки слід виконувати послідовно. Однак при додаванні нових перевірок код стає перевантаженим умовними операторами, що ускладнює підтримку.

Рішення:

Створити для кожної перевірки окремий клас з методом, який виконує потрібні дії. Далі всі об'єкти-обробники об'єднуються в ланцюг, де кожен обробник має посилання на наступного. Запит передається першому обробнику ланцюга, який або самостійно обробляє його, або передає далі. Якщо обробник не може виконати дію, запит продовжує передаватися далі по ланцюгу, аж поки не знайдеться відповідний обробник або ланцюг закінчиться.

Приклад з життя:

У JavaScript події в браузерах можна обробляти на різних рівнях DOM-дерева. Наприклад, для таблиці з рядками, кожен з яких має кнопку для видалення, можна поставити обробник не на кожну кнопку, а на tbody, що містить усі рядки. Якщо подія виникає на кнопці, але вона не має обробника, подія "піднімається" по ланцюгу: від кнопки до рядка, потім до tbody, що дозволяє зручно обробляти події динамічно.

Переваги:

- Зменшує залежність між клієнтом і обробниками.
- Відповідає принципу єдиного обов'язку.
- Підтримує принцип відкритості/закритості.

Недоліки:

- Запит може залишитися без обробки, якщо жоден обробник його не обробить.

Шаблон "Prototype"

Призначення:

Шаблон "Prototype" використовується для створення об'єктів шляхом копіювання існуючого шаблонного об'єкта. Це дозволяє спрощувати процес створення об'єктів, коли їх структура заздалегідь відома, та уникати прямого створення нових екземплярів.

Проблема:

Якщо потрібно скопіювати об'єкт, звичайний спосіб — створити новий об'єкт і вручну копіювати його поля. Але деякі частини об'єкта можуть бути приватними, що ускладнює доступ до них і порушує інкапсуляцію.

Рішення:

Шаблон "Prototype" доручає самим об'єктам реалізовувати метод `clone()`, який повертає їх копію. Це дозволяє створювати нові об'єкти без прив'язки до їхніх конкретних класів, залишаючи логіку копіювання всередині класу.

Приклад з життя:

У виробництві перед масовим випуском виготовляються прототипи, що дозволяють протестувати виріб. Ці прототипи служать шаблонами і не беруть участі в подальшому виробництві.

Переваги:

- Дозволяє клонувати об'єкти без прив'язки до конкретного класу.
- Зменшує дублювання коду ініціалізації.
- Прискорює процес створення об'єктів.
- Є альтернативою підкласам при створенні складних об'єктів.

Недоліки:

- Складність клонування об'єктів, що містять посилання на інші об'єкти.

Реалізація не менше 3-х класів відповідно до обраної теми:

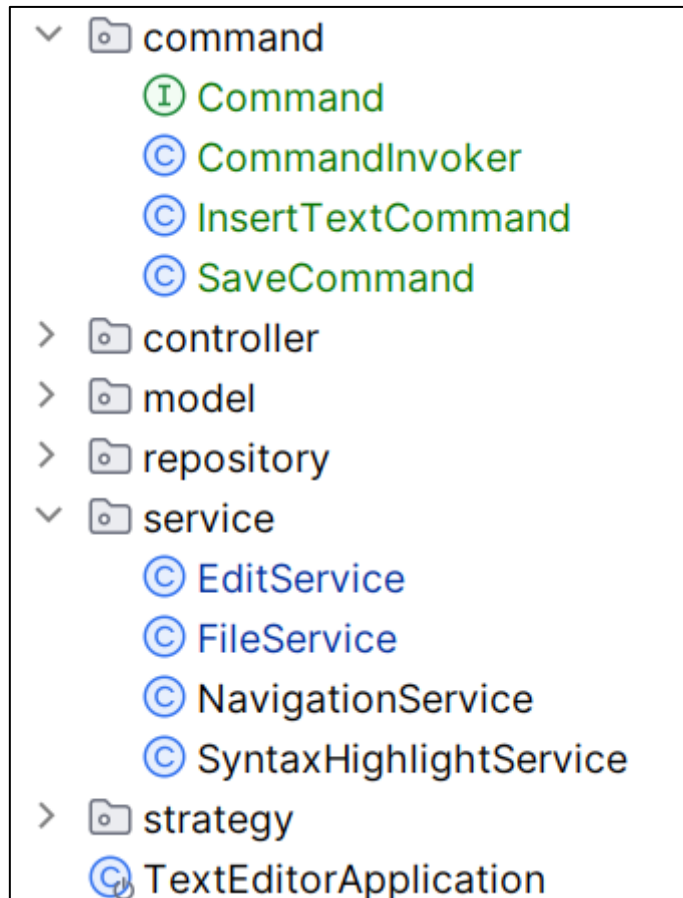


Рисунок 1. Структура проекту з використанням патерну «Команда».

У ході лабораторної роботи була реалізована система текстового редактора, основним завданням якої було використання патерну Команда для організації роботи з файлами та розширення функціоналу текстового редактора. Структура проєкту наведена на Рисунок 1. У результаті виконання роботи були реалізовані наступні кроки:

1. Розроблено структуру проєкту:

- Проєкт поділено на логічні пакети: controller, model, repository, service, strategy та command, що забезпечує зручність масштабування та підтримки коду.

2. Реалізовано патерн "Команда":

- Створено загальний інтерфейс Command, який визначає метод execute().
- Розроблено команди:
 - InsertTextCommand — команда для вставки тексту.
 - SaveCommand — команда для збереження файлу.
- Реалізовано клас CommandInvoker, який забезпечує виконання команд.

3. Реалізовано сервісний рівень:

- FileService відповідає за роботу з файлами (відкриття, збереження).
- EditService обробляє операції з текстом (вставка тексту, редагування).

4. Створено контролери для роботи з функціоналом:

- FileController — обробляє запити, пов'язані з файлами (відкриття, збереження, редагування).

5. Перевірено функціонал системи:

- Тестування API підтвердило коректність виконання команд (відкриття, збереження та редагування файлів).
- Система спроектована так, щоб її можна було легко розширити, додаючи нові функції без змін у наявному коді.

Цей підхід забезпечує зручність у розширенні функціоналу текстового редактора, дозволяючи легко додавати нові можливості, такі як збереження файлів чи вставка тексту. Використання патерну "Команда" спрощує організацію коду та забезпечує можливість впровадження додаткових функцій, покращуючи зручність для користувачів.

Реалізація шаблону за обраною темою:

1. Command



```
1 package com.example.text_editor.command;
2
3 public interface Command {
4     void execute();
5 }
```

Рисунок 2. Інтерфейс Command.

Цей інтерфейс, який ми бачимо на Рисунку 2, визначає базовий метод execute(), що реалізується всіма командами. Інтерфейс використовується для забезпечення єдиного підходу до виконання операцій незалежно від конкретної команди.

2. CommandInvoker

```
1 package com.example.text_editor.command;
2
3 import org.springframework.stereotype.Component;
4
5 import java.util.ArrayList;
6 import java.util.List;
7
8 5 usages new *
9 @Component
10 public class CommandInvoker {
11     2 usages
12     private final List<Command> commandHistory = new ArrayList<>();
13
14     3 usages new *
15     public void executeCommand(Command command) {
16         command.execute();
17         commandHistory.add(command);
18     }
19
20     no usages new *
21     public List<Command> getCommandHistory() { return commandHistory; }
```

Рисунок 3. Клас CommandInvoker.

Цей клас, який ми бачимо на Рисунку 3, відповідає за виконання команд та ведення їх історії. Він використовує метод `executeCommand (Command command)` для виконання команд і додає їх у список історії. `CommandInvoker` дозволяє відокремити виклик команд від їхньої реалізації, що робить систему гнучкішою.

3. InsertTextCommand

```
1 package com.example.text_editor.command;
2
3 import com.example.text_editor.service.EditService;
4
5 1 usage new *
6 public class InsertTextCommand implements Command {
7     2 usages
8     private final EditService editService;
9     2 usages
10    private final String currentContent;
11    2 usages
12    private final String newText;
13
14    1 usage new *
15    public InsertTextCommand(EditService editService, String currentContent, String newText) {
16        this.editService = editService;
17        this.currentContent = currentContent;
18        this.newText = newText;
19    }
20
21    1 usage new *
22    @Override
23    public void execute() { editService.insertText(currentContent, newText); }
```

Рисунок 4. Клас InsertTextCommand.

Цей клас, який ми бачимо на Рисунок 4, реалізує інтерфейс Command і відповідає за вставку тексту до існуючого вмісту. Використовує метод execute(), який викликає сервіс EditService для додавання нового тексту. InsertTextCommand дозволяє виконувати операцію вставки незалежно від зовнішнього контексту.

4. SaveCommand

```
1 package com.example.text_editor.command;
2
3 import com.example.text_editor.service.FileService;
4
5 4 usages new *
6 public class SaveCommand implements Command {
7     2 usages
8     private final FileService fileService;
9     2 usages
10    private final String fileName;
11    2 usages
12    private final String content;
13
14    2 usages new *
15    public SaveCommand(FileService fileService, String fileName, String content) {
16        this.fileService = fileService;
17        this.fileName = fileName;
18        this.content = content;
19    }
20
21    1 usage new *
22    @Override
23    public void execute() {
24        fileService.saveFile(fileName, content);
25    }
26 }
```

Рисунок 5. Клас SaveCommand.

Цей клас, який ми бачимо на Рисунок 5, реалізує інтерфейс Command і відповідає за збереження файлу. У методі execute() SaveCommand викликає сервіс FileService для збереження змін у файлі. Завдяки цьому класу забезпечується узгоджений підхід до операції збереження..

5. FileController

```
1 package com.example.text_editor.controller;
2
3 import com.example.text_editor.command.CommandInvoker;
4 import com.example.text_editor.command.SaveCommand;
5 import com.example.text_editor.model.TextFile;
6 import com.example.text_editor.service.FileService;
7 import org.springframework.web.bind.annotation.*;
8
9  * Vika *
10 @RestController
11 @RequestMapping("/file")
12 public class FileController {
13
14     4 usages
15     private final FileService fileService;
16     2 usages
17     private final CommandInvoker commandInvoker;
18
19     * Vika *
20     public FileController(FileService fileService, CommandInvoker commandInvoker) {
21         this.fileService = fileService;
22         this.commandInvoker = commandInvoker;
23     }
24
25     * Vika *
26     @GetMapping("/open")
27     public TextFile openFile(@RequestParam String fileName) {
28         return fileService.openFile(fileName);
29     }
30 }
```

Рисунок 6.1. Клас FileController.

```
31
32
33     new *
34     @PostMapping("/save")
35     public String saveFile(@RequestParam String fileName, @RequestParam String content) {
36         SaveCommand saveCommand = new SaveCommand(fileService, fileName, content);
37         commandInvoker.executeCommand(saveCommand);
38         return "File saved successfully.";
39     }
40
41     new *
42     @PutMapping("/edit")
43     public String editFile(@RequestParam String fileName, @RequestParam String newContent) {
44         TextFile file = fileService.editFile(fileName, newContent);
45         return "File edited successfully: " + file.getName();
46     }
47 }
```

Рисунок 6.2. Клас FileController.

Цей контролер, який ми бачимо на Рисунках 6.1 та 6.2, обробляє HTTP-запити, пов'язані з файлами. Він використовує методи:

- `openFile()` — відкриває файл за вказаною назвою.

- `saveFile()` — викликає команду `SaveCommand` для збереження файлу.
 - `editFile()` — редагує вміст файлу через сервіс `FileService`.
- Контролер координує роботу з файлами і є точкою доступу для клієнтів через API.

6. EditService

```
1 package com.example.text_editor.service;
2
3 import org.springframework.stereotype.Service;
4
5 6 usages  1 Vika *
6 @Service
7 public class EditService {
8
9     1 usage  1 Vika *
10     public String insertText(String currentContent, String newText) { return currentContent + "\n" + newText; }
11 }
```

Рисунок 7. Контролер EditService.

Цей клас, який ми бачимо на Рисунку 7, відповідає за реалізацію бізнес-логіки редагування тексту. Метод `insertText(String currentContent, String newText)` додає новий текст до існуючого, вставляючи його з нового рядка. Цей сервіс використовується для підтримки функціональності редагування тексту в системі, забезпечуючи зручність і простоту інтеграції.

7. FileService

```
1 package com.example.text_editor.service;
2
3 import com.example.text_editor.model.TextFile;
4 import com.example.text_editor.repository.TextFileRepository;
5 import org.springframework.stereotype.Service;
6
7 9 usages  👤 Vika *
8 @Service
9 public class FileService {
10
11     7 usages
12     private final TextFileRepository textFileRepository;
13
14     👤 Vika
15     public FileService(TextFileRepository textFileRepository) {
16         this.textFileRepository = textFileRepository;
17     }
18
19     1 usage  👤 Vika *
20     public TextFile openFile(String fileName) {
21         TextFile file = textFileRepository.findByName(fileName);
22         if (file != null) {
23             return file;
24         }
25         TextFile newFile = new TextFile(fileName, content: "Empty content");
26         return textFileRepository.save(newFile);
27     }
28 }
```

Рисунок 8.1. Контролер FileService.


```

1 usage  Vikar
25      public TextFile editFile(String fileName, String newContent) {
26          TextFile file = textFileRepository.findByName(fileName);
27          if (file != null) {
28              file.setContent(newContent);
29              return textFileRepository.save(file);
30          }
31          throw new RuntimeException("File not found");
32      }
33
34      1 usage  new *
35      public TextFile saveFile(String fileName, String content) {
36          TextFile file = textFileRepository.findByName(fileName);
37          if (file == null) {
38              file = new TextFile(fileName, content);
39          } else {
40              file.setContent(content);
41          }
42          return textFileRepository.save(file);
43      }

```

Рисунок 8.2. Контролер FileService.

Цей клас, який ми бачимо на Рисунках 8.1 та 8.1, відповідає за операції з файлами.

- `openFile(String fileName)` — відкриває файл або створює новий, якщо файл не існує.
- `editFile(String fileName, String newContent)` — змінює вміст файлу.
- `saveFile(String fileName, String content)` — зберігає зміни у файлі.

FileService інкапсулює логіку роботи з файлами, забезпечуючи простоту підтримки та масштабованість.

Посилання на код на GitHub: https://github.com/vvvvvvvvvviktoriia/text_editor

Діаграма класів для патерну «Command»:

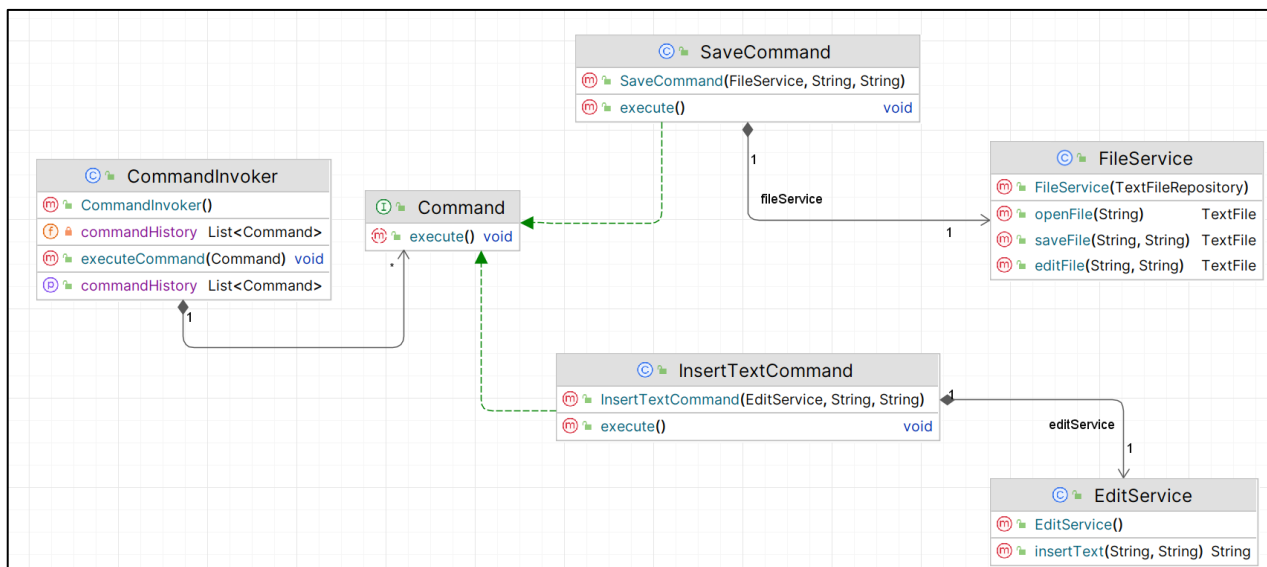


Рисунок 8. Діаграма класів для патерну «Command».

Раніше всі операції з файлами та текстом могли бути реалізовані безпосередньо у контролерах або сервісах, що призвело б до:

- Ускладнення підтримки коду через жорстку залежність між викликами операцій і логікою їх виконання.
- Неможливості зберігати історію виконаних дій для майбутньої реалізації функціоналу "Undo".
- Порушення принципу відкритості/закритості, оскільки додавання нових операцій вимагало б змін в існуючих класах.

Патерн «Команда» вирішив цю проблему, дозволивши:

- Інкапсулювати кожну операцію (збереження, вставку тексту тощо) у вигляді окремого об'єкта-команди.
- Організувати зберігання виконаних команд для можливості реалізації скасування дій.
- Зменшити залежність між викликами операцій та їх реалізацією.

Переваги застосування патерну «Command»:

1. Гнучкість:

- Легко додавати нові команди (наприклад, для додаткових операцій з файлами чи текстом) без змін в існуючих класах.

2. Принцип відкритості/закритості:

- Існуючі класи не потрібно змінювати для додавання нових функціональностей.

3. Читаємість і підтримка:

- Логіка кожної команди ізольована, що спрощує тестування та розуміння коду.

4. Можливість скасування дій:

- Збереження історії виконаних команд дозволяє реалізувати функціонал "Undo".

5. Модульність:

- Команди працюють незалежно, що дозволяє їх легко оновлювати чи замінювати без впливу на решту системи.

Висновок: у даній лабораторній роботі я реалізувала патерн «Command» для текстового редактора, що дозволило створити гнучкий механізм виконання операцій із файлами та текстом. Завдяки цьому патерну, кожна операція (збереження, вставка тексту тощо) була винесена в окремі класи, що спростило підтримку та розширення функціональності.

Переваги застосування патерну «Command» включають ізоляцію логіки виконання команд, зменшення залежностей між компонентами, можливість скасування виконаних дій та спрощення масштабування системи. Реалізація цього патерну зробила систему більш модульною, зрозумілою та легкою для розширення в майбутньому.